Chapter 9 On-policy Control with Approximation

本章探讨了在引入**函数近似 (function approximation)** 时的控制问题。具体来说，我们关注的是**同策略 (on-policy)** 设定，即：智能体用来生成行为（探索环境）的策略，也就是它正在学习和优化的策略。

本章首先介绍了 **半梯度 Sarsa (semi-gradient Sarsa)** 算法，这是将“半梯度 TD（时序差分）”算法从预测状态价值自然扩展到预测动作价值，并应用于同策略控制的结果。随后，内容推进到了 **n 步 Sarsa (n-step Sarsa)**。最后还提到了一个针对持续性任务非常有用的新设定：平均奖励 (average reward)。

半梯度 Sarsa (Semi-gradient Sarsa)

要得到一个“控制”算法，我们需要将预测算法从评估**状态价值** s_t 扩展到评估**动作价值** (s_t, a_t)。
我们不再使用 $s_t \to u_t$ 这样的更新形式，而是采用更广泛的形式：$(s_t, a_t) \to u_t$。
这里的**更新目标** u_t 可以是对真实动作价值 $q_\pi(s_t, a_t)$ 的任何近似。

将单步 TD 算法进行扩展，我们得到了新的更新目标（公式 9.1）：

$$u_t = r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}_t)$$

因此，单步 Sarsa 的权重更新规则可以写为（公式 9.2）：

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \alpha(r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}_t) - \hat{q}(s_t, a_t, \mathbf{w}_t)) \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w}_t)$$

为了将其变成一个控制方法（即能找到最优策略的方法），我们需要将这种动作价值的更新与**策略改进**技术结合起来。最直接的方法是基于当前的估计价值，采用ε**-贪婪 (**ε**-greedy) 策略**来选择动作

通过这种方式，智能体可以根据自己当前策略 π 生成的多个“幕 (episodes)”来不断优化该策略。这种方法被称为**幕式半梯度单步 Sarsa (episodic semi-gradient one-step Sarsa)**。

半梯度 n 步 Sarsa (Semi-gradient n-step Sarsa)

接下来，我们推导半梯度 Sarsa 的 n 步版本。我们使用 **n 步回报** 作为更新目标（公式 9.3）：

$$g_{t:t+n} = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n \hat{q}(s_{t+n}, a_{t+n}, \mathbf{w}_{t+n-1})$$

如果 $t+n \geq T$（即 n 步之后已经超出了这一幕的最终步数 T），我们就让 $g_{t:t+n} = g_t$（即从 t 时刻到游戏结束的真实总回报）。
基于此，n 步的更新公式为（公式 9.4）：

$$\mathbf{w}_{t+n} \leftarrow \mathbf{w}_{t+n-1} + \alpha(g_{t:t+n} - \hat{q}(s_t, a_t, \mathbf{w}_{t+n-1}))\nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w}_{t+n-1})$$

幕式半梯度 Sarsa 伪代码解析

Algorithm 9.1 Episodic Semi-gradient Sarsa

```
Initialize weight vector w ∈ ℝ^d arbitrarily
for each episode do
    Draw initial state s_0 and action a_0 (with ε-greedy)
    for each time step t = 0, 1, 2, ... do
        Take action a_t, observe r_{t+1} and s_{t+1}
        if s_{t+1} is terminal then
            w ← w + α (r_{t+1} − q̂(s_t, a_t, w)) ∇q̂(s_t, a_t, w).
            Go to next episode
        else
            Choose action a_{t+1} (with ε-greedy)
            w ← w + α (r_{t+1} + q̂(s_{t+1}, a_{t+1}, w) − q̂(s_t, a_t, w)) ∇q̂(s_t, a_t, w).
        end if
    end for
end for
return w
```

目标：通过不断玩游戏（经历多个 episode），学习一个权重向量 $\mathbf{w}$，从而能准确评估在状态 s 下采取动作 a 的价值，进而制定好策略。

- 任意初始化权重向量 $\mathbf{w} \in \mathbb{R}^d$
- **对每一个 episode（一局游戏）循环:**
  - 获取初始状态 s_0，并使用 ε-贪婪策略选择一个初始动作 a_0
  - **对每一步 $t = 0, 1, 2, \ldots$ 循环:**
    - 执行动作 a_t，观察到获得的奖励 r_{t+1} 和下一个状态 s_{t+1}
    - **如果 s_{t+1} 是终止状态 (游戏结束):**

- 更新权重: $\mathbf{w} \leftarrow \mathbf{w} + \alpha(r_{t+1} - \hat{q}(s_t, a_t, \mathbf{w}))\nabla\hat{q}(s_t, a_t, \mathbf{w})$ *(注意：游戏结束了，所以未来价值为 0，目标仅为 r_{t+1})*
- 进入下一个 episode

▪ **否则 (游戏继续):**
- 使用 ε-贪婪策略在 s_{t+1} 下选择下一个动作 a_{t+1}
- 更新权重: $\mathbf{w} \leftarrow \mathbf{w} + \alpha(r_{t+1} + \gamma\hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}) - \hat{q}(s_t, a_t, \mathbf{w}))\nabla\hat{q}(s_t, a_t, \mathbf{w})$

(注：如果在算法 9.1 的基础上换用 n 步回报的更新规则，就得到了幕式半梯度 n 步 Sarsa 算法。)

PS.之前的概念防止忘记：

函数近似 (Function Approximation)
• **概念**：以前学强化学习，我们用一个巨大的表格（Q-table）记住每一个"状态-动作"对的分数。但现实世界状态太多了（比如围棋的棋盘状态），表格根本存不下。函数近似就是用一个数学模型（通常是**神经网络**或者线性公式）去"猜"这个分数。
• **举个栗子**：去一个陌生的城市找市中心。你不需要背下城市里每一个十字路口该怎么走（表格法）。你只需要学到一个规律："只要朝着最高的电视塔走，大概率就能到市中心"（这就是函数近似，你提取了特征并拟合了规律，可以举一反三）。

同策略 (On-policy)
• **概念**：智能体**一边按照现有的策略自己做动作，一边根据这些动作的结果来优化这个策略**。
• **举个栗子**：学骑自行车。同策略就是你自己跨上自行一边摔跤一边调整平衡感。相对的，异策略（Off-policy）就像是你看着专业赛车手怎么骑，然后自己凭空总结一套完美的理论。

半梯度 (Semi-gradient)
• **概念**：在深度学习中，更新参数通常用梯度下降法求导。在强化学习的公式里，目标值 u_t 里包含着 $\mathbf{w}$，当前估计值 $\hat{q}$ 也包含 $\mathbf{w}$。所谓的"半"梯度，是指我们在求导时，**只对当前估计值 $\hat{q}$ 求导，而假装目标值 u_t 里的 $\mathbf{w}$ 是一个固定的常数，不去对它求导**。这是一种为了计算效率和算法稳定性所做的妥协。

n步 Sarsa (n-step Sarsa)
• **概念**：单步 Sarsa 只看眼前的一步（走一步，看获得多少奖励，然后马上猜未来的收益）。n步 Sarsa 是让你**连续走 n 步**，拿到真实的 n 步奖励后，再去猜剩下的未来收益。
• **举个栗子**：下国际象棋。单步 Sarsa 就像是一个只看下一步的新手，吃掉对方一个兵就觉得赚了；n步 Sarsa 像是一个能往后推演 3 到 4 步的高手，他会综合这 3-4 步的整体得失来评估当前的局面。

平均奖励：一种新的问题设定 (Average Reward: A New Problem Setting)

本节介绍了一种新的设定——**平均奖励 (Average reward)**。它适用于**不使用折扣 (without using discounting，即没有 γ)** 的**持续性问题 (continuing problems)**。这是经典动态规划理论中最常用的主要设定之一。

为什么需要“平均奖励”？
以前我们学习的都是“幕式任务 (Episodic tasks)”（比如下棋，有明确的输赢结局）。对于这类任务，我们只要把每一步的奖励加起来就行。

但如果是**持续性任务**（比如控制服务器散热风扇，或者交通信号灯，永远不停止），如果我们直接把未来所有的奖励加起来，总和会趋于**无限大**，这就没法比较哪个策略更好了。

- **过去的解决办法**：引入折扣因子 $\gamma < 1$，让未来的奖励“贬值”，从而算出一个有限的价值。
- **现在的解决办法（平均奖励）**：我们不看总和了，我们看**“平均每一步能拿多少分”**。只要策略 A 平均每步拿 5 分，策略 B 平均每步拿 3 分，我们就说策略 A 更好。

在平均奖励设定下，策略 π 的好坏是基于它的**平均奖励率**来评估的。策略 π 的平均奖励记为 $r(\pi)$

公式 (9.5)：平均奖励的定义

$$r(\pi) \doteq \lim_{h\to\infty} \frac{1}{h} \sum_{t=1}^{h} \mathbb{E}[R_t | s_0, a_{0:t-1} \sim \pi]$$

- $r(\pi)$: 策略 π 下的真实平均奖励。
- $\lim_{h\to\infty} \frac{1}{h} \sum_{t=1}^{h}$: 把前 h 步的期望奖励加起来，除以 h 求平均，然后让 h 趋于无穷大（考察长远表现）。
- $\mathbb{E}[R_t|...]$: 在给定初始状态，且一直遵循策略 π 的情况下的期望奖励。

公式 (9.6)：利用马尔可夫决策过程 (MDP) 展开

$$r(\pi) = \sum_{s \in \mathcal{S}} \mu_\pi(s) \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a) r$$

- $\mu_\pi(s)$: **稳态分布 (Steady state distribution)**。通俗地说，就是当你按照策略 π 运行了无限长的时间后，你**停留在状态 s 的时间比例（概率）**。
- 这个公式的意思是：平均奖励 = $\sum$ (处于某个状态的概率 $\times$ 在该状态下采取各种动作的概率 $\times$ 采取该动作后得到的期望奖励)。

公式 (9.7) & (9.8)：在实践中如何估计平均奖励

由于我们不知道环境的真实物理模型，我们要靠**采样**来估计 $r(\pi)$，记为 $\bar{r}_t$。

$$\bar{r}_t = \frac{1}{t} \sum_{k=1}^{t} r_k \quad (9.7)$$

通常我们使用**滑动平均 (running average)** 来增量式更新这个估计值：

$$\bar{r}_{t+1} = \bar{r}_t + \beta(r_{t+1} - \bar{r}_t) \quad (9.8)$$

$\bar{r}_t$: 我们对平均奖励的**当前估计值**。

β: 一个很小的正数，作为更新估计奖励的学习率。

幕式半梯度 n 步 Sarsa

Algorithm 9.2 Episodic Semi-gradient n-step Sarsa

```
Initialize weight vector w ∈ ℝ^d arbitrarily
for each episode do
    Draw initial state s_0 and action a_0 (with ε-greedy)
    Let T ← ∞
    for each time step t = 0, 1, 2, ... do
        if t < T then                                          ▷ next action available
            Take action a_t, observe r_{t+1} and s_{t+1}
            if s_{t+1} is terminal then
                Let T ← t + 1
            else                                               ▷ non-terminal, choose next action
                Choose action a_{t+1} (with ε-greedy)
            end if
        end if
        Let τ = t − n + 1
        if τ ≥ 0 then                                          ▷ Compute g and update for (s_τ, a_τ)
            Let g = Σ_{i=τ+1}^{min(τ+n,T)} γ^{i−τ−1} r_i
            if τ + n < T then
                Let g ← g + γ^n q̂(s_{τ+n}, a_{τ+n}, w)
            end if
            w ← w + α (g − q̂(s_τ, a_τ, w)) ∇q̂(s_τ, a_τ, w)
        end if
    end for
end for
return w
```

核心思想：不是走一步就更新，而是**连续走 n 步**。记录这 n 步的累积折扣奖励，加上第 n 步状态的预估价值，作为目标值 g，然后回头去更新第 1 步的"状态-动作"价值。伪代码中引入了时间戳 $\tau = t - n + 1$ 来追踪当前正在更新哪一步。

稳态分布与各态历经性 (Ergodicity)

公式 (9.9)：稳态分布的定义

$$\mu_\pi(s) \doteq \lim_{t\to\infty} \Pr(S_t = s | a_{0:t-1} \sim \pi)$$

公式 (9.10)：稳态分布的平衡方程

$$\mu_\pi(s') = \sum_{s\in\mathcal{S}} \mu_\pi(s) \sum_a \pi(a|s) p(s'|s,a)$$

- **通俗解释**：流入状态 s' 的概率总和，等于状态 s' 本身的稳态概率。达到了一种动态平衡。
- **各态历经性 (Ergodic)**
  - 书里假设 MDP 是 **ergodic (各态历经/遍历的)**。这意味着：无论你从哪个状态 s_0 开始，只要时间足够长，你到达各个状态的概率分布都会稳定在同一个 μ_π。

- **举个栗子**：想象一个连通的地铁网。不管你第一天是从“机场站”上车还是从“火车站”上车，只要你每天在里面随机乱坐（策略 π），一年之后，你在“市中心站”被发现的概率是一个固定的值（比如 15%）。初始起点的影响随着时间推移完全消失了。

差分回报 (Differential Return)

在平均奖励设定下，我们不再关心绝对奖励累加，而是关心**每一小步的奖励比“平均水平”好多少还是差多少**。

公式 (9.11)：差分回报

$$g_t \doteq r_{t+1} - r(\pi) + r_{t+2} - r(\pi) + r_{t+3} - r(\pi) + \dots$$

- $r_t - r(\pi)$: 称为**差分奖励**。
- **通俗解释**：假设你的策略平均每步能赚 10 块钱 ($r(\pi) = 10$)。如果你这一步赚了 12 块，你的差分奖励就是 +2（超常发挥）；如果只赚了 8 块，就是 -2（发挥失常）。我们现在的目标是最大化这些**“超额利润”的总和**。
- 基于差分回报定义的价值函数被称为**差分价值函数 (differential value functions)**，记法依然是 $v_\pi(s)$ 和 $q_\pi(s,a)$。

Algorithm 9.3 Differential semi-gradient Sarsa for continuing task

Initialize weight vector $\mathbf{w} \in \mathbb{R}^d$ arbitrarily
Initialize reward estimate, *e.g.* $\bar{r} = 0$
Initialize state s and action a
for each step **do**
  Take action a, observe r and s'
  Choose a' following π (ε-greedy *w.r.t.* $\hat{q}$)
  $\bar{r} \leftarrow \bar{r} + \beta(r - \bar{r})$
  $\mathbf{w} \leftarrow \mathbf{w} + \alpha(r - \bar{r} + \hat{q}(s', a', \mathbf{w}) - \hat{q}(s, a, \mathbf{w}))\nabla q(s, a, \mathbf{w})$
  $s \leftarrow s'$
  $a \leftarrow a'$
end for
Return $\mathbf{w}$

针对持续性任务的差分半梯度 Sarsa

这是本章的核心算法实践。

核心差异（与传统的 Sarsa 相比）：

1. **没有折扣因子** γ：在更新公式中，γ 消失了（相当于默认为 1）。
2. **使用差分奖励计算 TD 误差**：

传统的 TD 误差：$r + \gamma\hat{q}(s', a') - \hat{q}(s, a)$

现在的 TD 误差：$r - \bar{r} + \hat{q}(s', a', \mathbf{w}) - \hat{q}(s, a, \mathbf{w})$

3. **双轨并行学习**：不仅要学习权重 $\mathbf{w}$（更新动作价值），还要同时更新对平均奖励的估计 $\bar{r}$。

修改后的贝尔曼方程 (Modified Bellman Equations)
在平均奖励设定下，经典的贝尔曼方程也发生了变形，只需**把原本的即时奖励 r，替换成差分奖励** $(r - r(\pi))$ 即可。

公式 (9.12) & (9.13)：差分状态价值与动作价值方程

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\big((r - r(\pi)) + v_\pi(s')\big)$$

$$q_\pi(s,a) = \sum_{s',r} p(s',r|s,a) \left((r - r(\pi)) + \sum_{a'} \pi(a'|s') q_\pi(s',a') \right)$$

公式 (9.14) & (9.15)：差分贝尔曼最优方程

最优策略不仅要最大化动作价值，也要最大化整体的平均奖励 $\max_\pi r(\pi)$。

$$v_*(s) = \max_a \sum_{s',r} p(s',r|s,a)\big((r - \max_\pi r(\pi)) + v_*(s')\big)$$

$$q_*(s,a) = \sum_{s',r} p(s',r|s,a) \left((r - \max_\pi r(\pi)) + \max_{a'} q_*(s',a') \right)$$

差分半梯度方法 (Differential Semi-gradient Methods)

核心思想：我们不再使用传统的“折扣回报 (discounted return, 带有 γ)”，而是使用**差分回报 (differential return)** 来推导控制算法的修改版本。这意味着算法在评估价值时，关注的是“比平均水平好多少”，而不是“绝对总收益是多少”。

差分半梯度 Sarsa (Differential semi-gradient Sarsa)
为了更新我们的模型（即更新权重 $\mathbf{w}$），我们首先需要计算**时序差分误差 (TD Error, 记为 δ_t)**。在差分设定下，TD 误差的计算公式也发生了相应的变化。

差分形式的 TD 误差

- **公式 (9.16)：针对状态价值函数 (State-value function, v)**

$\delta_t = r_{t+1} - \bar{r}_{t+1} + \hat{v}(s_{t+1}, \mathbf{w}_t) - \hat{v}(s_t, \mathbf{w}_t)$
(注：这是预测问题中用来评估处于某个状态好坏的误差公式)

- **公式 (9.17)：针对动作价值函数 (Action-value function, q)**

$\delta_t = r_{t+1} - \bar{r}_{t+1} + \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}_t) - \hat{q}(s_t, a_t, \mathbf{w})$
(注：这就是控制算法 Sarsa 实际用到的误差公式，因为它需要评估具体动作的好坏来做决策)

符号与概念通俗拆解：

- δ_t (Delta)：**TD 误差**。它代表了“现实带来的惊喜（或惊吓）”。如果 $\delta_t > 0$，说明这一步的结果比预期的好；如果 $\delta_t < 0$，说明比预期的差。
- $r_{t+1} - \bar{r}_{t+1}$：这是**即时差分奖励**。r_{t+1} 是你刚刚真实拿到的奖励，$\bar{r}_{t+1}$ 是系统当前对“平均每步能拿多少奖励”的估计。它表示你这一步是**“超常发挥”**还是**“发挥失常”**。
- $\hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}_t)$：这是模型对**下一步**的价值预估（未来潜力）。
- $\hat{q}(s_t, a_t, \mathbf{w})$：这是模型在走这一步之前，对**当前步**的价值预估（原本的期望）。
  总结： `差分 TD 误差 = (超额奖励 + 下一步的预估潜力) - 原本对当前步的期望`。注意，这里**没有折扣因子** γ！

权重更新规则

公式 (9.18)：使用差分 TD 误差的半梯度更新公式

$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \alpha \cdot \delta_t \cdot \nabla_{\mathbf{w}} \hat{q}(s_t, a_t, \mathbf{w}_t)$
💡 **公式拆解：**
• 这个公式告诉你**如何修正你的神经网络（或数学模型）**。
• **新权重 = 老权重 + 学习率 $\times$ 误差 $\times$ 梯度（调整方向）**。
• 如果 δ_t 是正的（结果比预期好），我们就顺着梯度 $\nabla_{\mathbf{w}}$ 的方向稍微调整一点参数 $\mathbf{w}$，让模型下次在同样状态下给出更高的评分。由此，我们就得到了上一节中展示的完整算法（算法 9.3）。

差分半梯度 n 步 Sarsa (Differential semi-gradient n-step Sarsa)

单步 Sarsa 只看眼前的一步，我们同样可以把它推广到 **n 步 (n-step)** 的情况，也就是让智能体连续走 n 步，把这 n 步的“超额奖励”都累加起来，再结合第 n 步之后的预估潜力，作为我们更新的“目标靶子”。

n 步差分回报目标

公式 (9.19)：n 步更新目标 $g_{t:t+n}$

$$g_{t:t+n} \doteq r_{t+1} - \bar{r}_{t+1} + r_{t+2} - \bar{r}_{t+2} + \cdots + r_{t+n} - \bar{r}_{t+n} + \hat{q}(s_{t+n}, a_{t+n}, \mathbf{w}_{t+n-1})$$

概念通俗拆解：

- **连续的考核**：想象你在考察一个员工的表现。单步 Sarsa 是每天考核一次；n 步 Sarsa 是连续考察他 n 天，把这 n 天里他**每天超出或低于公司平均业绩的部分** $(r - \bar{r})$ 全部加起来，再加上对他未来潜力的评估 $\hat{q}(\ldots)$。
- **边界条件 ($t + n \geq T$)**：和之前一样，如果考察还没到 n 天，这局游戏/任务就结束了（到达了终止状态 T），那么我们就不需要加上未来的预估 $\hat{q}$ 了，直接定义 $g_{t:t+n} \doteq g_t$（即从时间 t 开始到结束的真实累积差分回报）。